
Parcours coach et client

Le code de FitFlow, écran par écran et requête par requête

« Chaque étape d'un parcours utilisateur, jusqu'au fichier, à la route et à la requête SQL qui l'exécute. »

3 applications

25 modules d'API

133 endpoints

36 modèles

Document de préparation à la soutenance

Marc YRIUS — relevé dans le code le 3 septembre 2026

Complément du dossier de projet · ne fait pas partie du dossier remis

— Comment lire ce document

Objet. Le dossier de projet explique *ce que fait* FitFlow et *pourquoi* les décisions ont été prises. Ce document-ci explique *comment le code l'exécute*, pas à pas, sur les deux parcours qui portent le produit : celui du coach et celui du client. Il est fait pour être révisé avant la soutenance et pour tenir une question technique précise sans hésiter.

Provenance. Tout ce qui suit a été lu dans le code le 3 septembre 2026 — pas repris du dossier. Chaque étape cite son fichier et sa ligne d'entrée. Là où le code contredit le dossier, c'est signalé sur place et récapitulé en partie 9 : mieux vaut le savoir avant qu'un examinateur ouvre le dépôt.

Convention de lecture d'une étape

Chaque étape d'un parcours se lit sur cinq niveaux, toujours dans le même ordre :

Écran

le fichier de l'interface, et le geste de l'utilisateur

Appel

la fonction du client HTTP, et la requête réellement émise

Route

le module Express et les middlewares traversés

Controller

la fonction serveur, ses contrôles et sa requête Prisma

Données

les tables touchées, et les effets de bord

1 Vue d'ensemble en une page

1.1 Trois applications, un seul serveur de vérité

Application	Technologie	Rôle dans les parcours
Dashboard web fitflow-dashboard	Next.js 16 (App Router), React 19, TypeScript, Tailwind 4	Parcours coach complet ; parcours client en version allégée ; écrans d'administration
Application mobile coach-mobile-app	React Native 0.81 sur Expo 54, JavaScript	Parcours client en salle ; et aussi un parcours coach — voir partie 6
API backend-coach-app	Node 22, Express 4, Prisma 5, PostgreSQL	Toute la logique métier, l'autorisation, les tâches planifiées

Les deux clients ne partagent aucun code : ils partagent un **contrat**. Chacun a son propre module d'appel HTTP — `fitflow-dashboard/lib/api.ts` et `coach-mobile-app/src/services/api.js` — qui expose les mêmes ressources avec les mêmes chemins. C'est la conséquence directe de la décision 1 du dossier : aucune règle métier n'est dupliquée côté client.

1.2 Les couches, du geste à la ligne SQL

```
// Exemple : le client coche une série pendant sa séance
SessionDetailScreen.js      toggleSet(exerciseId, setNumber, exercise)
↓
services/api.js             setCompletionsAPI.update({...})
↓                             intercepteur : Authorization = Bearer <JWT du cache mémoire>
```

```

↓
PUT /api/set-completions    helmet → compression → cors → json → apiLimiter
↓
routes/setCompletions.js    authenticate → authorize('CLIENT','COACH')
↓
setCompletions.controller    canAccessExercise(userId, exerciseId) → 403 sinon
↓
Prisma                      setCompletion.upsert({ where: { exerciseId_setNumber } })
↓
PostgreSQL                  INSERT ... ON CONFLICT (exercise_id, set_number) DO UPDATE

```

Chaîne complète — Les six niveaux sont les mêmes pour les 133 endpoints. Retenir cet enchaînement permet de répondre à « que se passe-t-il quand... » sur n'importe quelle fonctionnalité, sans avoir mémorisé chaque écran.

1.3 Ordre exact des middlewares — `src/server.js`

#	Middleware	Ce qu'il fait, et pourquoi il est à cette place
1	<code>app.set('trust proxy', 1)</code>	Production seulement. Sans lui, <code>req.ip</code> vaut l'adresse du proxy Fly.io et le limiteur compte tous les visiteurs comme un seul client
2	<code>helmet()</code>	En-têtes de sécurité. <code>crossOriginResourcePolicy: false</code> parce que <code>/uploads</code> sert des images consommées depuis l'origine du dashboard
3	<code>compression()</code>	gzip sur toutes les réponses. Posé avant les routes, sinon il ne couvrirait rien. Gain mesuré : 7 934 → 2 186 octets sur une recherche d'exercices
4	<code>cors()</code>	Origine <code>*</code> en développement, liste blanche <code>FRONTEND_URL</code> en production
5	<code>express.json({ limit: '10mb' })</code>	Corps JSON. La limite haute existe pour les photos encodées
6	<code>express.static('uploads')</code>	Pilote local du stockage de fichiers. En production, les fichiers vont dans Supabase Storage et ce dossier ne sert plus
7	<code>apiLimiter</code> sur <code>/api</code>	Limiteur général. Le limiteur strict est posé route par route sur <code>/login</code> et <code>/register</code>
8	25 modules de routes	<code>/api/auth</code> en premier, puis modération, puis le métier
9	<code>import './jobs/...'</code>	Les trois tâches <code>node-cron</code> démarrent à l'import du serveur, dans le même processus

2 Le socle commun aux deux parcours

2.1 Authentification — les trois routes qui ouvrent tout

Route	Controller	Ce qui s'y joue
POST <code>/api/auth/register</code>	<code>authController.register</code> (l. 9)	Validation manuelle : champs requis, format d'email par expression régulière, mot de passe ≥ 6 caractères, rôle limité à COACH ou CLIENT . Unicité de l'email vérifiée avant écriture. Mot de passe haché par bcrypt (10 tours) puis <code>user.create</code>
POST <code>/api/auth/login</code>	<code>authController.login</code> (l. 105)	<code>findUnique({ where: { email } })</code> → <code>bcrypt.compare</code> → <code>generateToken({ userId, role })</code> . Même message d'erreur dans les deux cas d'échec

GET /api/auth/me	authController.getMe (l. 202)	Renvoie l'utilisateur <i>et son profil métier</i> — c'est ce que les deux clients appellent au démarrage pour savoir s'il faut passer par l'onboarding
------------------	-------------------------------	--

Le jeton. `utils/jwt.js` ne fait que deux choses : `jwt.sign(payload, secret, { expiresIn })` et `jwt.verify`. La charge utile contient `userId` et `role` — rien d'autre, et surtout aucune donnée personnelle : un JWT n'est pas chiffré, seulement signé. Algorithme HS256, durée 30 jours, secret en variable d'environnement.

À savoir avant la question. Il n'y a **ni refresh token, ni liste de révocation** : un jeton volé reste valable jusqu'à son échéance, et la déconnexion ne fait que l'effacer côté client. C'est écrit en 9.5 du dossier. La réponse à donner est le correctif : jeton d'accès court plus refresh token, ou liste de révocation en base.

2.2 Les deux middlewares d'accès — `src/middlewares/auth.js`

```
// authenticate : charge l'utilisateur ET statue sur son statut de modération
const user = await chargerUtilisateur(req); // Bearer → verifyToken → findUnique
if (!user) return sendError(res, 'Authentification requise', 401);

if (user.status === 'SUSPENDED') {
  const termeEchu = user.suspendedUntil && new Date(user.suspendedUntil) <= new Date();
  if (termeEchu) { /* réactivation à la volée */ }
  else return res.status(403).json({ moderation: { reason, until, appealEndpoint } });
}
req.user = user; next();
```

Deux conséquences à retenir — 1) `authenticate` fait **une requête en base à chaque appel authentifié** : il ne se contente pas de vérifier la signature. C'est le prix du statut de modération, et c'est ce que montre la figure 11 du dossier. 2) Une suspension échue est levée **au premier accès**, pas par une tâche planifiée qui balaierait la table.

`authorize(...roles)` est l'autre moitié : il ne connaît que `req.user.role` et renvoie 403 si le rôle n'est pas dans la liste. Une troisième porte existe, `authenticateEvenIfSuspended`, utilisée par **une seule route** — `POST /api/appeals` — pour qu'une sanction n'emporte pas le droit de la contester.

2.3 Autorisation par appartenance — `src/utls/authorization.js`

Le rôle dit ce qu'on est, pas ce qu'on possède. Cinq fonctions centralisent la seconde couche, et chaque controller les appelle avant d'écrire :

Fonction	Règle exacte
<code>resolveProfiles(userId)</code>	Charge en parallèle le profil coach et le profil client. Un utilisateur n'a que l'un des deux
<code>canAccessClient(userId, clientId)</code>	Vrai si c'est le client lui-même, ou un coach lié par une relation <code>ClientCoach</code> <code>isActive: true</code>
<code>canAccessConversation(userId, coachId, clientId)</code>	Vrai pour les deux participants, personne d'autre
<code>canAccessProgram(userId, programId)</code>	Charge le programme, compare au profil coach propriétaire ou au profil client assigné
<code>canAccessSession / canAccessExercice</code>	Remontent la chaîne <code>Exercice → Session → Program</code> pour retomber sur la règle précédente

C'est cette couche qu'une campagne de tests a fait naître : `__tests__/authorization.test.js` construit deux écosystèmes complets et vérifie 18 refus et 11 accès légitimes. La distinction rôle / appartenance est l'étude de cas 17.2 du dossier.

2.4 Le jeton côté client — deux stockages, une seule logique

	Mobile — <code>services/api.js</code>	Dashboard — <code>lib/api.ts</code>
Stockage	<code>expo-secure-store</code> (trousseau iOS / Keystore Android), avec repli <code>localStorage</code> si <code>Platform.OS === 'web'</code>	<code>localStorage</code>
Lecture	Cache mémoire <code>_tokenCache</code> : le trousseau n'est lu qu'une fois. Lire <code>SecureStore</code> en arrière-plan est interdit sur iOS	<code>localStorage.getItem</code> à chaque requête, protégé par <code>typeof window !== "undefined"</code> — sans quoi le rendu serveur de Next.js planterait
Sur 401	Purge le stockage, appelle un callback qui remet le contexte React à zéro. Un drapeau <code>_unauthorizedHandled</code> évite dix déconnexions pour dix requêtes parallèles	Purge puis <code>window.location.href = "/login"</code> , sauf sur <code>/auth/login</code> et <code>/auth/register</code> — sinon un mot de passe faux provoquerait une redirection au lieu d'un message
Délai maximal	<code>timeout: 15000</code> — une API injoignable doit échouer explicitement plutôt que laisser un écran en chargement sans fin	Aucun délai configuré

L'ordre de la déconnexion, et pourquoi il compte

Côté mobile, `logout()` appelle `setTokenCache(null)` **avant** de purger le trousseau. L'inverse laisserait une fenêtre pendant laquelle une requête concurrente lirait encore le jeton du compte précédent depuis la mémoire — et l'enverrait au serveur. C'est un bug de session croisée en puissance, évité par deux lignes dans le bon ordre.

2.5 Le contrat de réponse

```
// Succès - utils/responseHandler.js
{ success: true, message: "Opération réussie", data: { ... } }
// Erreur
{ success: false, message: "Description de l'erreur", errors: null }
```

Les deux clients lisent donc toujours `response.data.data`. C'est ce qui permet aux intercepteurs de traiter les erreurs de façon uniforme, et c'est aussi ce qui explique cette double indirection qu'un examinateur peut relever dans le code des écrans.

3 Parcours coach — étape par étape

3.1 Inscription puis onboarding

Niveau	Détail
Écran	<code>app/auth/register/page.tsx</code> — formulaire avec choix du rôle
Appel	<code>authAPI.register({ email, password, firstName, lastName, role: 'COACH' })</code>
Route	POST <code>/api/auth/register</code> — <code>authLimiter</code> puis <code>controller</code> , sans <code>authenticate</code>
Controller	Crée le <code>User</code> avec le mot de passe haché, renvoie <code>{ user, token }</code>
Données	Table <code>users</code> uniquement. Aucun profil métier n'existe encore — c'est ce qui rend l'onboarding obligatoire

Le contexte React stocke le jeton, puis la navigation constate l'absence de `coachProfile` et envoie vers `/coach/onboarding`. L'écran soumet `onboardingAPI.submitCoach(payload)` → PUT `/api/coaches/onboarding`, qui crée le `CoachProfile` (bio, spécialités, ville, lieux d'entraînement).

Le piège du contexte périmé — c'est l'étude de cas 17.4 du dossier. À la première connexion, l'objet utilisateur en mémoire n'a pas de profil ; après l'onboarding, la base en a un mais le contexte l'ignore encore, et toute logique de navigation renvoie l'utilisateur à l'onboarding. La solution est `refreshUser()` : on redemande GET `/auth/me` et **le serveur fait autorité**, plutôt que de reconstruire l'objet localement.

3.2 Tableau de bord — trois appels en parallèle

```
// app/(dashboard)/coach/dashboard/page.tsx
const [programmes, clients, rdv] = await Promise.all([
  programsAPI.getCoachPrograms(), // GET /api/programs/coach
  clientsAPI.getCoachClients(),   // GET /api/clients/coach
  appointmentsAPI.getUpcoming(),  // GET /api/appointments/upcoming
]);
```

`Promise.all` et non trois `await` successifs : les trois requêtes sont indépendantes, les enchaîner tripleraient la latence perçue. Les indicateurs de tête (clients, programmes actifs, messages non lus, taux de complétion) sont calculés **côté client** à partir de ces trois réponses — il n'y a pas d'endpoint de synthèse.

`getCoachClients` (`clients.controller.js` l. 6) part du `CoachProfile` de l'appelant, puis lit la table de liaison `ClientCoach` avec `isActive: true`. C'est le point qui a fait tomber l'écran Analytics : deux endpoints de la même page lisaient deux sources différentes du même lien.

3.3 Accepter une demande — la seule transaction du parcours coach

```
// requests.controller.js - acceptRequest (l. 192)
const existingCoaches = await prisma.clientCoach.findMany({
  where: { clientId: request.clientId, isActive: true },
});
const isPrimary = existingCoaches.length === 0; // premier coach = référent

await prisma.$transaction([
  prisma.coachRequest.update({ where: { id: requestId }, data: { status: 'accepted' } }),
  prisma.clientCoach.upsert({ where: { clientId_coachId: { ... } }, ... }),
  ...(isPrimary ? [prisma.clientProfile.update({ data: { coachId: coachProfile.id } })] : []),
]);
```

Trois choses à savoir défendre ici. 1) L'`upsert` plutôt qu'un `create` : une relation désactivée puis réacceptée se réactive, sans violer la contrainte d'unicité `@unique([clientId, coachId])`. 2) Le premier coach devient automatiquement référent. 3) La troisième écriture alimente encore `clientProfile.coachId`, le champ hérité de l'ancien modèle `un-coach` — voir l'écart n° 4 en partie 9.

3.4 Créer un programme, puis son calendrier

Étape	Route et effet
Nouveau programme <code>coach/clients/[clientId]/program/new</code>	POST <code>/api/programs</code> — vérifie que le client est bien lié au coach, crée Program (titre, description, durée, <code>isActive</code>)
Calendrier <code>coach/programs/[programId]/calendar</code>	GET <code>/api/sessions/program/:programId</code> — liste les séances datées ; le calendrier place une carte par jour et ouvre l'éditeur au clic

3.5 L'éditeur de séance — le cœur de l'interface coach

Le fichier de page ne fait que dix lignes : il délègue à `app/(dashboard)/coach/programs/[programId]/sessions/_components/session-editor.tsx`, 1 033 lignes, partagé entre la création et l'édition.

```
// Capteurs : souris ET clavier - c'est ce qui rend le glisser-déposer accessible
const sensors = useSensors(
  useSensor(PointerSensor, { activationConstraint: { distance: 5 } }),
  useSensor(KeyboardSensor, { coordinateGetter: sortableKeyboardCoordinates }),
);

const handleDragEnd = (event: DragEndEvent) => {
  const { active, over } = event;
  if (!over || active.id === over.id) return;
  setExercises((prev) => arrayMove(prev, oldIndex, newIndex)); // état local
};
```

Point technique	Ce qu'il faut pouvoir expliquer
<code>activationConstraint: { distance: 5 }</code>	Un déplacement ne démarre qu'après 5 pixels. Sans ce seuil, un simple clic sur une carte serait interprété comme un début de glissement et les champs deviendraient difficiles à cliquer
<code>KeyboardSensor</code>	Le réordonnement est opérable au clavier. C'est mesuré en 11.4 du dossier — et c'était donné pour « non vérifié » dans une version antérieure
Ordre = index du tableau	Le champ <code>order</code> n'est pas manipulé pendant le glissement : il est recalculé à l'enregistrement, <code>order: i</code> sur la position finale. Moins d'états à synchroniser
Rien n'est persisté au glissement	Le coach essaie plusieurs agencements sans une seule requête réseau. La sauvegarde est explicite : <i>Brouillon</i> (DRAFT) ou <i>Enregistrer</i> (DONE)
Validation avant envoi	<code>isValidReps</code> refuse les répétitions non numériques et liste les exercices fautifs par leur position : « Exercice 3 (Burpees) : « douze » ». Un message qui désigne la ligne vaut mieux qu'un refus global

Ce que fait le serveur à l'enregistrement

```
// sessionController.js - upsertSession (l. 8)
// 1. le coach est-il propriétaire du programme ?
if (program.coachId !== coachProfile.id) return sendError(res, 'Accès non autorisé...', 403);

// 2. la séance existe-t-elle ? par id, sinon par la clé (programId, date)
existingSession = id ? findUnique({ where: { id } })
  : findUnique({ where: { programId_date: { programId, date } } });

// 3. mise à jour : les exercices sont REMPLACÉS
await prisma.exercise.deleteMany({ where: { sessionId: session.id } });
if (exercises.length > 0) await prisma.exercise.createMany({ data: ... });
```

Deux conséquences que je préfère connaître avant qu'on me les demande.

- a) Les performances du client sont perdues.** `SetCompletion` est en `onDelete` : Cascade sur `Exercise` : supprimer les exercices efface les séries déjà validées. Si le coach modifie une séance que le client a commencée, le réalisé disparaît. Correctif : réconcilier les exercices par identifiant au lieu de tout remplacer.
- b) Ce n'est pas atomique.** Le `deleteMany` puis `createMany` ne sont pas dans un `$transaction` — un échec entre les deux laisse une séance sans exercices. Le projet sait le faire ailleurs : les rendez-vous, la modération et les disponibilités utilisent `prisma.$transaction`. C'est un oubli, pas un choix.

La clé composite (`programId`, `date`) est l'autre pièce à connaître : elle rend l'enregistrement idempotent. Deux envois du même formulaire sans identifiant ne créent pas deux séances le même jour — c'est la contrainte

@unique([programId, date]) qui l'interdit, et l'upsert qui l'exploite.

3.6 Agenda et rendez-vous — le morceau le plus dense du back

Détection de conflit

```
// appointment.controller.js - checkConflict (l. 12)
const conflict = await prisma.appointment.findFirst({
  where: {
    status: 'CONFIRMED',
    OR: [{ coachId }, ...(clientId ? [{ clientId }] : [])],
    startAt: { lt: endAt }, // le test de chevauchement standard :
    endAt: { gt: startAt }, // A commence avant la fin de B ET finit après le début de B
  },
});
```

Deux inégalités suffisent à détecter tout chevauchement, dans les deux sens. Seuls les rendez-vous **CONFIRMED** bloquent : une proposition en attente ne réserve pas le créneau — c'est un choix, et il se défend (sinon une proposition jamais traitée gèlerait l'agenda).

Le 409 n'est pas une fin de non-recevoir

Cas	Réponse du serveur
Conflit, sans résolution demandée	409 avec le détail du RDV en conflit (id, titre, début, fin, durée) — le front peut alors proposer un choix à l'utilisateur au lieu d'afficher « échec »
<code>resolution: 'replace'</code>	Supprime le RDV existant, et ses occurrences filles si c'était une série
<code>resolution: 'shorten'</code>	Raccourcit celui des deux qui commence le premier, et recalcule <code>durationMinutes</code>

Récurrence : RRULE, et une transaction à trois écritures

```
// Occurrences générées à la création, bornées à un an
const rule = new RRule({ ...RRule.parseString(rruleStr), dtstart: startDate });
const dates = rule.all((date) => date <= oneYearLater);

await prisma.$transaction(async (tx) => {
  const root = await tx.appointment.create({ data: { ..., rrule } }); // la racine
  await tx.appointment.createMany({ data: dates.map(... parentId: root.id) }); // les occurrences
  const proposal = await tx.message.create({ data: { type: 'APPOINTMENT_PROPOSAL', ... } });
});
```

Les occurrences sont matérialisées en base, pas calculées à la lecture. Avantage : une occurrence peut être annulée ou déplacée individuellement, et l'agenda se lit par une simple requête de plage. Coût : un an de récurrence hebdomadaire crée 52 lignes, et changer la règle exige de régénérer. Le format RRULE (RFC 5545) est le même que celui des agendas du marché — les récurrences restent interopérables.

Les clients, eux, ne peuvent pas créer de série : `if (rrule) return sendError(res, 'Les clients ne peuvent pas créer de RDV récurrents', 403)`. Et un client bloqué par le coach (`CoachClientBlock`) ne peut rien proposer.

Confirmer : la règle relationnelle

C'est l'étude de cas 17.2, et le code est explicite : le coach ne peut confirmer que ce que le client a proposé, et réciproquement. L'information « qui a proposé » n'est pas sur le rendez-vous mais sur le **message lié**, via `message.isSentByCoach` — d'où l'impossibilité de trancher la règle dans un middleware, qui ne connaît que le rôle et le jeton.

Le seul test manquant du projet est ici. Cette règle est vérifiée à la main sur les quatre combinaisons, pas par un test automatisé. C'est le marqueur [À COMPLÉTER] de 13.4, et la dette que je citerais moi-même si on me demandait ce que je ferais autrement.

3.7 Messagerie coach

Mécanisme	Implémentation
Rafraîchissement	<code>setInterval(fetchMessages, 3000)</code> dans <code>coach/messages/page.tsx</code> (l. 264) — uniquement quand une conversation est ouverte , et l'intervalle est nettoyé au démontage
Compteur non lus	<code>components/layout/sidebar.tsx</code> l. 127 : rafraîchi toutes les 10 secondes côté dashboard
Isolation	<code>canAccessConversation(userId, coachId, clientId)</code> — une conversation n'appartient qu'à ses deux participants
Types de message	<code>CHAT</code> , <code>TIP</code> (conseil, affichage distinct), <code>APPOINTMENT_PROPOSAL</code> (lié 1-1 à un rendez-vous)

Le polling est le point faible assumé du produit : une conversation ouverte dix minutes émet 200 requêtes dont la quasi-totalité ne renvoie rien. C'est le gisement principal de la partie 12 du dossier et la deuxième évolution prioritaire. La réponse à préparer : WebSocket ou SSE, et pourquoi ça n'a pas été fait — un canal persistant demande un serveur qui garde l'état des connexions, ce qui change la nature du déploiement.

3.8 Analytics — quatre calculs à savoir défendre

Endpoint	Calcul	Le détail qui compte
GET <code>/analytics/workout/volume</code>	Volume = poids × répétitions , cumulé par semaine ISO et par groupe musculaire	Le volume d'un exercice est attribué en entier à chacun de ses <code>bodyParts</code> : un développé couché compte à 100 % pour les pectoraux et pour les épaules. Simplification assumée — pondérer demanderait un modèle de contribution musculaire
GET <code>/analytics/workout/estimated-1rm</code>	Brzycki si répétitions ≤ 10, Epley au-delà, arrondi à 0,5 kg	Le dossier et le cahier des charges annoncent « formule d'Epley » : c'est incomplet, il y a deux formules et un seuil. Brzycki est plus juste sur les faibles répétitions, Epley ne s'effondre pas sur les séries longues
GET <code>/analytics/workout/completion</code>	Taux de complétion et série de séances consécutives	Ne compte que les séances réellement validées par le client
INOL, standards de force, repères de volume	<code>utils/strengthStandards.js</code> , <code>utils/volumeLandmarks.js</code>	Trois endpoints supplémentaires, alimentés par des tables de référence — c'est ce qui donne du sens aux chiffres bruts

Pourquoi les données réelles sont des chaînes de caractères, et ce que ça coûte

`SetCompletion.repsAchieved` et `weightUsed` sont des `String`? : en salle, un client écrit « 12-15 », « AMRAP », « 80kg », « échec ». Un champ numérique aurait refusé la saisie ou perdu l'information.

Le prix est payé au moment du calcul, dans `utils/parseSetData.js` : `parseNumericField` gère les plages (« 12-15 » → 15 par défaut, stratégie `high/low/avg`), les unités (« 80kg », « 100lbs »), les valeurs non numériques (→ `null`, la série est ignorée du calcul). **42 tests unitaires** couvrent cette seule fonction — c'est la suite la plus dense du projet, et c'est justifié : elle est en amont de tous les indicateurs.

Et un champ séparé, `durationAchieved`, existe précisément pour que les 45 secondes d'un gainage ne soient jamais comptées comme 45 répétitions. Le mobile applique la même exclusivité à l'envoi : `repsAchieved: chronometre ? undefined : repsAchieved`.

4 Parcours client — étape par étape

Le parcours client vit sur le mobile. Le dashboard en propose une version allégée (`app/(dashboard)/client/...`), mais c'est l'application Expo qui porte la promesse produit : saisir ses séries pendant l'effort, une main occupée.

4.1 Inscription et onboarding en trois étapes

Écran	Ce qui est demandé, et ce qui part au serveur
<code>RegisterScreen.js</code>	Email, mot de passe, nom, prénom, rôle <code>CLIENT</code> → <code>POST /api/auth/register</code>
<code>onboarding/client/Step1Physical.js</code>	Genre, âge, taille, poids
<code>Step2Location.js</code>	Ville, salle de sport (recherche dans <code>gyms</code>)
<code>Step3Goal.js</code>	Objectif et niveau
<code>ClientOnboardingScreen.js</code>	Assemble les trois étapes et soumet en une fois → <code>PUT /api/clients/onboarding</code> → <code>submitClientOnboarding</code> crée le <code>ClientProfile</code>

`StepProgress.js` matérialise l'avancement. Le parcours est obligatoire mais un mode « passer » existe (exigence F1.5) : le profil peut être complété plus tard, au prix d'un dashboard sans recommandations.

4.2 Trouver un coach et envoyer une demande

```
// CoachSearchScreen.js → GET /api/coaches?city=...
// coaches.controller : ne renvoie que les profils visibleInSearch, et un select explicite
include: {
  user: { select: { firstName: true, lastName: true } }, // email volontairement exclu
  reviews: { select: { rating: true } },
}
```

C'est l'extrait 4 du dossier, et l'argument de sécurité à retenir : **l'email ne fuite pas dans une réponse publique** parce que le `select` est explicite, pas parce qu'on a pensé à le retirer après coup.

Puis `CoachDetailScreen.js` → `requestsAPI.create({ coachId, message })` → `POST /api/requests`. Le message d'accompagnement est **obligatoire** (F16.2), un doublon en attente est refusé (F16.6), et le refus conserve la demande au statut `rejected` pour l'audit (F16.5).

4.3 L'accueil mobile — `DashboardScreen.js`, 1 421 lignes

Ce qui se charge, et dans quel ordre

```
// Au montage : deux appels en parallèle
const [programsRes, meRes] = await Promise.all([
  programsAPI.getClientPrograms(), // GET /api/programs/client - programmes + séances + exercices
  authAPI.getMe(),                 // pour récupérer clientProfile.id
]);
// puis, seulement si le client n'a aucun coach :
if (!hasCoach) await onboardingAPI.getRecommendedCoaches();

// À chaque changement de date sélectionnée : deux appels ciblés
useEffect(() => { loadStats(); loadMeals(); }, [date, clientId]);
```

Un seul appel ramène tout l'entraînement. GET /api/programs/client renvoie les programmes avec leurs séances et leurs exercices imbriqués ; la frise de dates n'émet donc aucune requête quand on change de jour pour l'entraînement. Le suivi quotidien et les repas, eux, sont datés : ils se rechargent par jour.

La séance du jour est choisie côté client

```
const getActiveProgram = () => programs.filter(p => p.isActive)
  .sort((a, b) => new Date(b.createdAt) - new Date(a.createdAt))[0] || null;

const getSessionForDate = (dateStr) => (getActiveProgram()?.sessions || [])
  .find(s => format(new Date(s.date), 'yyyy-MM-dd') === dateStr) || null;
```

Deux décisions visibles dans quatre lignes. S'il existe plusieurs programmes actifs — cas réel du multi-coach — c'est **le plus récent** qui s'affiche : un choix arbitraire mais déterministe, préférable à un écran vide. Et la comparaison se fait sur une date *formatée en heure locale*, pas sur les objets Date : comparer des instants ferait basculer la séance d'un jour selon le fuseau.

L'enregistrement automatique du suivi — le détail que je citerais spontanément

```
// Enregistrement 900 ms après la dernière frappe
useEffect(() => {
  if (vientDuServeur.current) { vientDuServeur.current = false; return; }
  if (minuterieRef.current) clearTimeout(minuterieRef.current);
  minuterieRef.current = setTimeout(enregistrerStats, 900);
  return () => clearTimeout(minuterieRef.current);
}, [formData, clientId, date]);
```

Garde-fou	Le problème qu'il évite
Temporisation de 900 ms	Sans elle, chaque caractère du champ de notes part en requête — et deux réponses arrivées dans le désordre écrasent la plus récente
<code>vientDuServeur</code> (<code>useRef</code>)	Le formulaire vient d'être peuplé par le serveur : sans ce drapeau, l'enregistrement automatique se déclencherait au chargement et réécrirait les données avec ce qu'il vient de lire. Un <code>useRef</code> et non un <code>useState</code> : changer ce drapeau ne doit pas provoquer de rendu
<code>setFormData</code> (prev => ...)	Forme fonctionnelle obligatoire : deux frappes rapprochées liraient sinon le même état périmé et la première serait perdue

Le tout finit sur POST /api/stats → `statController`, un **upsert par (clientId, date)** : la journée est une ligne unique qu'on met à jour, pas un historique d'événements.

4.4 Le détail de séance — `SessionDetailScreen.js`, 1 053 lignes

C'est l'écran le plus dense du produit et celui qu'il faut connaître par cœur : il porte la promesse de FitFlow.

a) Valider une série

```
// toggleSet (l. 261) — un basculement, deux directions
if (isCompleted) {
  await setCompletionsAPI.delete(exerciseId, setNumber); // DELETE → décocher
} else {
  const chronometre = estChronometre(exercise);
  await setCompletionsAPI.update({
    exerciseId, setNumber,
    repsAchieved: chronometre ? undefined : repsAchieved, // exclusifs :
    durationAchieved: chronometre ? durationAchieved : undefined, // jamais les deux
    weightUsed: chronometre ? undefined : weightUsed,
    rpe: rpe || undefined,
  });
}
```

```
if (restTimerEnabled && exercise.restTime) startRestTimer(parseRestTime(exercise.restTime));
}
```

Les valeurs par défaut sont **les valeurs prescrites** : cocher sans rien modifier enregistre ce que le coach avait prévu. C'est ce qui rend la validation possible en une tape, et c'est aussi ce qui distingue le prévu du réalisé : `Exercise` porte la prescription, `SetCompletion` la performance.

Côté serveur, `updateSetCompletion` est un **upsert sur la clé composite (`exerciseId`, `setNumber`)** : corriger une charge met la ligne à jour au lieu d'en créer une seconde. C'est l'extrait 5 du dossier, et la contrainte `@unique` qui le permet.

b) Le chronomètre de repos, et la fonction qui l'a réparé

```
// parseRestTime - le champ restTime est libre, et plusieurs écritures coexistent en base
'1:30'      → 90      // minutes:secondes
'1min30'    → 90      // avec unité, secondes optionnelles
'2min'      → 120
'90s' / '90' → 90      // un nombre nu se lit en secondes
```

Pourquoi cette fonction existe — l'histoire à raconter

L'implémentation précédente ne lisait que le format `m:ss` et renvoyait `0` pour tout le reste — c'est-à-dire **pour la totalité des valeurs réellement enregistrées**. Résultat : le chronomètre de repos ne démarrait jamais, et la cause n'était pas dans le chronomètre mais dans la lecture de son entrée.

C'est un exemple concret de dette de conception : un champ texte libre côté coach oblige à une tolérance côté client. L'alternative — contraindre la saisie à un format unique — se défend aussi, mais aurait rejeté les données existantes.

Deux chronomètres cohabitent, et il faut ne pas les confondre : le **chrono de séance** (`elapsedSeconds`, avec pause et reprise, visible en haut de l'écran) et le **chrono de repos** (`restSecondsLeft`, compte à rebours avec barre de progression et bouton « Passer », qui remplace la barre du premier pendant le repos). Le repos utilise un **horodatage absolu de fin** (`restEndTimeRef`) et non un décompte : une application mise en arrière-plan puis rouverte retrouve le temps juste, ce qu'un compteur décrémente aurait perdu. Une notification locale (`expo-notifications`) annonce « Repos terminé ! 🍌 ».

c) La fin de séance n'est pas automatique — et il faut le dire ainsi

```
// Le pied d'écran est piloté par un booléen calculé côté client
<Text>{completed} / {total} séries terminées</Text>
<TouchableOpacity disabled={!allCompleted}>
  {allCompleted ? 'Valider la séance' : 'Complétez toutes les séries'}
</TouchableOpacity>

// Puis, sur confirmation explicite :
await sessionsAPI.validateSession(sessionId, durationSeconds); // PUT /sessions/:id/validate
```

Écart avec le dossier, à connaître absolument. Le dossier (8.5) et le cahier des charges (F5.4) annoncent que « la séance se marque automatiquement comme `DONE` quand tous les sets sont complétés », et l'extrait 10 montre un calcul `allCompleted` côté serveur. **Ce calcul n'existe pas dans le backend** : `grep "status: 'DONE'"` ne renvoie qu'une seule occurrence, dans `validateSession`, appelée par le client.

Ce qu'il faut répondre : le mécanisme est réel mais son siège est le mobile — l'interface compte les séries, déverrouille le bouton et demande une confirmation. La formulation juste est « la séance se clôt sans que le coach n'ait rien à faire », pas « le serveur la clôt tout seul ». Et le choix se défend : une validation explicite évite de clore une séance qu'un client aurait cochée par erreur, et c'est elle qui transporte la durée réelle mesurée par le chrono.

4.5 Nutrition

Geste	Chaîne
Chercher un aliment	<code>FoodSearch.js</code> → GET <code>/api/food/search?q=...</code> → table <code>CiqualFood</code> (référentiel ANSES, alimenté par <code>scripts/seed-ciqual.js</code>)
Enregistrer un repas	<code>mealsAPI.create</code> → POST <code>/api/meals</code> — catégorie (petit-déjeuner, déjeuner, dîner, collation) et aliments
Voir ses macros	Calcul des calories, protéines, glucides et lipides par repas et par jour
Calculer ses besoins	<code>CalorieCalculator.js</code> → POST <code>/api/nutrition/calculate</code> — à partir du poids, de la taille, de l'âge, du niveau d'activité et de l'objectif

L'argument de conception : le référentiel est une **table de référence officielle** plutôt qu'une base saisie à la main. C'est une conséquence directe de la veille (16.2), et ça évite d'avoir à défendre la fiabilité de valeurs nutritionnelles inventées.

4.6 Agenda, messagerie et notifications côté client

Écran	Mécanisme
<code>AppointmentsScreen.js</code>	Liste et confirmation des propositions. Les sélecteurs de date et d'heure sont natifs (<code>@react-native-community/datetimestpicker</code>) — un argument concret pour la décision 2 : une web-app n'y a pas accès
<code>MessagesScreen.js</code>	<code>setInterval(fetchMessages, 3000)</code> (l. 186), uniquement conversation ouverte. Marquage « lu » à l'ouverture
<code>NotificationsBell.js</code>	<code>POLL_INTERVAL = 30000</code> (l. 17) → GET <code>/api/notifications/unread-count</code> . Badge plafonné à « 99+ ». Panneau en bottom-sheet, marquage individuel ou global
Onglet Messages	<code>AppNavigator.js</code> tient un second compteur, rafraîchi toutes les 30 s et à chaque retour sur l'onglet

L'endpoint le plus sollicité de l'API, et son cache

```
// notification.controller.js - getUnreadCount (l. 29)
const cached = await getUnreadCount(userId);
if (cached !== null) return sendSuccess(res, { count: cached, cached: true });

const count = await prisma.notification.count({ where: { userId, isRead: false } });
await setUnreadCount(userId, count);
sendSuccess(res, { count, cached: false });
```

Trois choses à savoir défendre : le champ `cached` est renvoyé **exprès**, pour rendre le mécanisme observable en démonstration ; l'invalidation se fait à l'écriture (`markAsRead` supprime la clé plutôt que de la recalculer) ; et si Redis est absent, `getRedis()` renvoie `null`, toutes les opérations de cache deviennent des non-opérations et l'application continue en interrogeant PostgreSQL. **Redis n'est pas déployé en production** — décision assumée après la correction de latence de 14.5.

5 Le mobile porte aussi un parcours coach

Fait non documenté dans le dossier. Le dossier présente l'application mobile comme « client uniquement » (6.2, figures 10 et 14). **C'est faux.** `AppNavigator.js` branche sur le rôle : `user.role === 'COACH' ? <CoachTabs /> : <MainTabs />`, et onze écrans coach existent sous `src/screens/coach/`.

Écran mobile coach	Ce qu'il permet
<code>CoachDashboardScreen.js</code>	Vue d'ensemble du portefeuille
<code>CoachClientsScreen.js</code> / <code>CoachClientDetailScreen.js</code>	Liste des clients et fiche détaillée
<code>CoachNewProgramScreen.js</code>	Création d'un programme
<code>CoachProgramCalendarScreen.js</code>	Calendrier du programme
<code>CoachSessionEditorScreen.js</code> (1 431 lignes)	Édition d'une séance — la plus grosse vue mobile
<code>CoachSessionFillScreen.js</code>	Remplir une séance <i>à la place</i> du client, et la valider
<code>CoachTemplatesScreen.js</code> , <code>CoachMessagesScreen.js</code> , <code>CoachProfileScreen.js</code> , <code>EditCoachProfileScreen.js</code>	Templates, messagerie, profil

Comment le présenter sans se contredire

Deux réponses honnêtes, à choisir selon la question :

Si on demande la cible produit : « Le mobile est *conçu* pour le client — c'est là qu'est la valeur, en salle. Un côté coach existe dans le code, mais il n'est pas la voie recommandée : le coach travaille sur le dashboard, parce que construire un programme sur plusieurs semaines demande un grand écran. »

Si on ouvre le dépôt et qu'on voit les onze écrans : le reconnaître immédiatement et expliquer la genèse — le dépôt s'appelle `coach-mobile-app`, le projet a d'abord visé le coach sur mobile, puis la contrainte d'usage a inversé la priorité. Les écrans coach sont restés, fonctionnels ; le dossier a décrit la cible et non l'état du code. **C'est une inexactitude du dossier, pas une invention du code.**

6 Les technologies, une par une

Technologie	Rôle exact dans FitFlow	Ce que je réponds si on demande « pourquoi celle-là ? »
Express 4	Serveur HTTP, 25 modules de routes, 133 endpoints	Le modèle de middlewares composables est exactement ce dont j'avais besoin : <code>authenticate</code> puis <code>authorize</code> puis contrôler, déclaré en une ligne par route. Fastify serait plus rapide, mais la performance de l'API n'a jamais été le facteur limitant — la latence venait du pooler de base
Prisma 5	ORM, schéma unique de 36 modèles, source des migrations	Un seul fichier fait office de modèle logique, de générateur de client typé et de source de migrations. Le schéma est la documentation — c'est ce qui permet de générer les figures 12 et 13 du dossier. Contrepartie : les requêtes complexes se pensent en Prisma, pas en SQL, et il faut connaître le coût de chaque <code>include</code>
PostgreSQL	Source de vérité, 36 tables, 48 relations, 37 cascades	Le modèle est intrinsèquement relationnel : un client a des coachs, qui ont des programmes, qui ont des séances, qui ont des exercices, qui ont des validations. Les contraintes d'unicité composites (<code>ClientCoach</code> , <code>SetCompletion</code> , <code>Review</code>) portent des règles métier que du code applicatif devrait sinon garantir

JWT (HS256)	Authentification sans état, 30 jours	Deux clients hétérogènes servis par un seul mécanisme, sans stockage de sessions partagé. HS256 et non RS256 parce qu'il n'y a qu'un seul émetteur et un seul vérificateur : une paire de clés asymétriques n'apporterait rien ici
bcrypt	Hachage des mots de passe, 10 tours	Fonction lente par construction, avec sel intégré, et <code>compare</code> à temps constant. Argon2 est aujourd'hui préférable ; bcrypt reste un choix défendable et largement audité
Next.js 16 App Router	Dashboard : routage par dossiers, 29 pages	Le routage par système de fichiers rend la structure des URL lisible, et les segments dynamiques (<code>[clientId]</code> , <code>[programId]</code>) évitent une table de routes à maintenir. Précision utile : 36 fichiers portent "use client" — les données sont chargées par le navigateur, pas par le serveur Next. Je n'utilise donc ni composants serveur pour les données, ni actions serveur
React 19	Interface du dashboard	Composants et hooks. Le compilateur React de cette version a d'ailleurs révélé deux vrais défauts au moment de réparer la chaîne d'intégration (15.3) : une lecture d'horloge pendant le rendu, et un <code>setState</code> synchrone dans un effet
TypeScript	Dashboard uniquement	Le build échoue sur une erreur de type : c'est une gate de plus dans la chaîne. Le mobile est en JavaScript — asymétrie assumée, conséquence d'un démarrage plus rapide sur ce dépôt
Tailwind 4	Style du dashboard, thème sombre	C'est la version 4 qui a rendu le thème sombre tenable : elle expose ses couleurs en variables <code>--color-*</code> , et surcharger 96 variables sous <code>[data-theme]</code> fait basculer 243 utilitaires d'un coup — y compris ceux des composants écrits après. C'est l'étude de cas 17.3
@dnd-kit	Réordonnancement des exercices	Deux capteurs, dont un capteur clavier : le glisser-déposer reste opérable sans souris. C'était le critère de choix face à des bibliothèques plus anciennes fondées sur HTML5 drag-and-drop, dont l'accessibilité est mauvaise
React Native 0.81 + Expo 54	Application mobile	Une base de code pour iOS et Android, mes compétences React réutilisées, et aucune configuration native à maintenir. Conséquence assumée : Expo Go interdit les notifications push système et HealthKit — ces deux fonctionnalités sont hors périmètre à cause de ce choix, et je le présente ainsi
expo-secure-store	Jeton sur mobile	Trousseau iOS / Keystore Android, plutôt qu' <code>AsyncStorage</code> qui écrit en clair. C'est une exigence du cahier des charges technique (T4.3)
axios	Client HTTP des deux applications	Les intercepteurs sont la raison du choix : un seul endroit ajoute le jeton, un seul endroit traite les 401. Avec <code>fetch</code> , cette logique serait dupliquée à chaque appel ou enveloppée à la main
node-cron	Trois tâches planifiées, dans le processus de l'API	Simple et suffisant : pas d'ordonnanceur externe à déployer. C'est aussi la raison du refus du serverless (décision 7) : sans processus permanent, le rappel H-1 toutes les 5 minutes devenait impossible
rrule	Récurrence des rendez-vous	Implémentation du standard RFC 5545. Écrire ma propre syntaxe de récurrence aurait fermé la porte à toute interopérabilité avec un agenda tiers
Redis	Cache du seul compteur de notifications	Le besoin est de mémoriser <i>une valeur volatile sous une clé, avec expiration</i> — c'est la définition d'un magasin clé-valeur. MongoDB aurait coché la case « NoSQL » sans répondre au problème. Non déployé : 10 \$/mois pour un gain que la mesure ne réclame plus

node:test	Tests backend, 168 tests	Runner natif de Node : une dépendance lourde en moins et aucune transpilation à configurer. C'est une conséquence directe de la veille technique (16.2)
Vitest + Testing Library	41 tests de composants	Vitest partage la configuration de build du projet. Testing Library impose de tester ce que l'utilisateur voit — un test qui cherche un texte à l'écran résiste à un remaniement interne
Playwright	5 tests de bout en bout	Le parcours d'authentification dans un vrai navigateur. Peu de tests, mais ils couvrent le chemin dont l'échec bloquerait tout le reste
helmet, compression, express-rate-limit	Défenses et sobriété du serveur	En-têtes de sécurité, gzip (72 % de réduction mesurée), et deux limiteurs — dont un strict sur <code>/login</code> et <code>/register</code> qui ne compte que les échecs , réglage trouvé après un effet de bord observé (16.3)

7 Vingt questions techniques, et ce que je réponds

7.1 Architecture et sécurité

Question	Réponse
« Que se passe-t-il si j'ouvre <code>/coach/dashboard</code> sans être connecté ? »	La page s'affiche une fraction de seconde puis redirige : la garde est côté client , dans <code>app/(dashboard)/layout.tsx</code> , et il n'y a pas de middleware.ts . Ce n'est pas un trou de sécurité — aucune donnée n'est dans le HTML, tout vient de l'API qui exige le jeton — mais c'est une garde d'ergonomie, pas de sécurité. Le correctif propre est un middleware Next qui vérifie la présence du jeton avant de rendre la route
« Pourquoi le jeton est-il en <code>localStorage</code> sur le web ? »	C'est la faiblesse n° 1 de mon tableau 9.5 : une faille XSS permettrait de l'exfiltrer. Un cookie <code>httpOnly + SameSite=Strict</code> est la bonne réponse, avec une politique de sécurité de contenu à défaut. Sur mobile, le problème ne se pose pas : le jeton est dans le trousseau système
« Un coach peut-il lire le programme d'un confrère ? »	Non, et c'est testé : <code>canAccessProgram</code> compare le profil coach de l'appelant au propriétaire du programme, et <code>__tests__/authorization.test.js</code> vérifie 18 tentatives de traversée qui doivent toutes renvoyer 403. Le second volet du test — 11 accès légitimes qui doivent renvoyer 200 — est indispensable : sans lui, un serveur qui refuse tout le monde passerait le premier
« Pourquoi une requête en base à chaque appel authentifié ? »	Parce que <code>authenticate</code> ne vérifie pas seulement la signature du jeton : il recharge l'utilisateur pour statuer sur son statut de modération. Un compte suspendu doit être arrêté immédiatement, pas à l'expiration de son jeton. C'est un coût assumé et mesuré : le trajet vers la base est de 19 ms
« Comment empêchez-vous quelqu'un de se déclarer administrateur ? »	La route d'inscription liste explicitement les rôles acceptés (<code>COACH</code> ou <code>CLIENT</code>) et cette garde a été conservée à l'ajout du rôle <code>ADMIN</code> . Il n'existe aucune route HTTP de promotion : un administrateur se crée en ligne de commande, ce qui suppose un accès au serveur

7.2 Données et cohérence

Question	Réponse
« Pourquoi les répétitions sont-elles des chaînes de caractères ? »	Parce qu'en salle un client écrit « 12-15 », « AMRAP » ou « échec ». Un entier aurait refusé la saisie. Le prix est payé au calcul, dans <code>parseNumericField</code> , couverte par 42 tests unitaires. Et <code>durationAchieved</code> est un champ séparé

	pour que 45 secondes de gainage ne soient jamais comptées comme 45 répétitions
« Que se passe-t-il si le coach modifie une séance déjà commencée ? »	Les exercices sont supprimés et recréés, et <code>SetCompletion</code> étant en <code>onDelete: Cascade</code> , les séries déjà validées disparaissent . C'est un défaut réel que j'ai identifié en préparant cette présentation. Le correctif est de réconcilier les exercices par identifiant plutôt que de remplacer le lot
« Vos écritures multiples sont-elles atomiques ? »	Là où ça compte, oui : la proposition de rendez-vous (RDV + message), la série récurrente (racine + occurrences + message), l'acceptation d'une demande (statut + relation), les actions de modération. Une exception : l'enregistrement d'une séance fait <code>deleteMany</code> puis <code>createMany</code> hors transaction. C'est un oubli, pas un choix, et il est réparable en trois lignes
« Comment évitez-vous les doublons ? »	Par des contraintes en base plutôt que par du code : <code>@@unique([clientId, coachId])</code> sur la relation, <code>@@unique([exerciseId, setNumber])</code> sur la validation d'une série, <code>@@unique([programId, date])</code> sur une séance, <code>@@unique([clientId, coachId])</code> sur un avis. Les <code>upsert</code> s'appuient dessus : c'est PostgreSQL qui garantit l'unicité, pas une vérification applicative qu'on pourrait oublier
« Pourquoi 13 index seulement, pour 36 tables ? »	Un index accélère la lecture et coûte à chaque écriture. Ils sont posés là où une requête liste et filtre sur une plage : agenda d'un coach sur une période, compteur de notifications non lues, file de modération, comptes dont la suppression est échue. Jamais sur une colonne servie par sa clé primaire, que PostgreSQL indexe déjà
« Le multi-coach, comment est-il modélisé ? »	Table de liaison <code>ClientCoach</code> porteuse d'attributs : <code>isPrimary</code> , <code>isActive</code> , dates de début et de fin. Une simple clé étrangère interdisait structurellement la fonctionnalité différenciante du produit — c'est l'étude de cas 17.1, et la migration a dû reprendre les données existantes

7.3 Temps réel, tâches planifiées, performance

Question	Réponse
« Pourquoi du polling et pas des WebSockets ? »	Parce que le polling fonctionne sans changer la nature du déploiement, et que la fenêtre est bornée : 3 secondes, uniquement quand une conversation est ouverte . Je connais le coût — 200 requêtes pour dix minutes de conversation — et c'est la deuxième évolution prioritaire du dossier. Un canal persistant exige un serveur qui garde l'état des connexions, incompatible avec la mise en veille automatique des machines qui ramène le coût à zéro
« Vos rappels de rendez-vous peuvent-ils partir en double ? »	Non : le champ <code>reminderSentAt</code> est le garde-fou. La requête ne sélectionne que les rendez-vous <code>CONFIRMED</code> dont le rappel est <code>null</code> , et l'écriture du champ les sort définitivement du périmètre. C'est de l'idempotence par la donnée, pas par un verrou
« Combien de tâches planifiées, et pourquoi dans l'API ? »	Trois : rappels J-1 à 8h00, rappels H-1 toutes les 5 minutes (fenêtre glissante de 55 à 65 minutes), purges de modération à 3h00. Elles vivent dans le processus de l'API — c'est ce qui a fait écarter le serverless, où aucun processus ne survit entre deux requêtes
« Pourquoi une fenêtre de 55–65 minutes et pas exactement une heure ? »	Parce que la tâche s'exécute toutes les 5 minutes : une égalité stricte raterait tous les rendez-vous tombant entre deux exécutions. Dix minutes de fenêtre couvrent largement l'intervalle, et <code>reminderSentAt</code> empêche le doublon que ce chevauchement pourrait produire
« Vos 300 ms de p95, vous les avez mesurés ? »	Oui, et le premier diagnostic était faux. J'avais attribué l'écart à la distance Paris–Irlande ; mesuré depuis la machine, le trajet vaut 19 ms . Les 70 ms restants venaient du pooler en mode transaction , conçu pour le sans-serveur

	et inutile sur une machine persistante. Après bascule en mode session : de 512 à 245 ms sur l'endpoint le plus lourd. C'est le passage du dossier que je trouve le plus instructif
« Pourquoi <code>Promise.all</code> partout ? »	Parce que ces requêtes sont indépendantes. Trois <code>await</code> successifs additionnent trois latences réseau ; en parallèle on paie la plus lente. C'est visible à l'accueil coach (3 appels) et à l'accueil client (2 appels)

7.4 Interface et qualité

Question	Réponse
« Votre glisser-déposer est-il accessible ? »	Oui, et c'est mesuré : <code>KeyboardSensor</code> avec <code>sortableKeyboardCoordinates</code> est actif dans les deux contextes concernés. En revanche l'accessibilité globale du produit est inégale — 19 % des champs du dashboard sont correctement étiquetés, et le mobile n'a quasiment aucun <code>accessibilityLabel</code> . Le correctif du dashboard est concentré sur un composant partagé : <code>useId()</code> et un <code>htmlFor</code> corrigeraient d'un coup tous les champs qui l'emploient
« Pourquoi 900 ms de temporisation sur le suivi quotidien ? »	Sans elle, chaque caractère du champ de notes part en requête et deux réponses arrivées dans le désordre écrasent la plus récente. Ce n'est pas du confort : c'est une protection contre une condition de course
« Comment testez-vous sans simulacres ? »	Les tests d'intégration tournent contre une vraie base PostgreSQL et une vraie API HTTP : simuler l'ORM revient à tester le simulacre. Les contraintes d'unicité, les cascades et le comportement réel des requêtes ne se vérifient qu'ainsi. Contrepartie assumée : ces tests exigent une base disponible, ils ne sont pas exécutables à froid
« Que vaut votre couverture ? »	214 tests dans 21 fichiers , sur quatre niveaux, détaillés en annexe D du dossier. Aucun rapport de couverture n'est produit — c'est une limite que j'assume, et je préfère parler de ce qui est couvert que d'un pourcentage. Le seul manque identifié est le test de la règle de confirmation d'un rendez-vous

8 Les écarts entre le dossier et le code

Sept écarts relevés en lisant le code pour ce document. Les connaître, c'est pouvoir les reconnaître avant qu'on les découvre.

#	Le dossier dit	Le code fait	Ce que je réponds
1	Application mobile « client uniquement » (6.2, figures 10 et 14)	<code>AppNavigator</code> branche sur le rôle ; 11 écrans coach existent et fonctionnent	Le mobile est <i>conçu</i> pour le client ; le côté coach est un héritage du démarrage du projet, resté fonctionnel. Le dossier a décrit la cible, pas l'état du code
2	La séance passe « automatiquement » en <code>DONE</code> quand toutes les séries sont cochées (8.5, F5.4, extrait 10)	Le mobile calcule <code>allCompleted</code> , déverrouille un bouton, et le client confirme → <code>PUT /sessions/:id/validate</code>	Le mécanisme est réel, son siège est le client. Formulation juste : « sans que le coach n'ait rien à faire ». La confirmation explicite transporte en plus la durée mesurée
3	1RM estimé « par la formule d'Epley » (glossaire, F10.3)	Brzycki si répétitions ≤ 10 , Epley au-delà	Deux formules et un seuil : Brzycki est plus juste sur les faibles répétitions, Epley ne s'effondre pas sur les séries longues. Les tests le montrent explicitement

4	<code>clientProfile.coachId</code> « n'est plus alimenté » (5.6, et un commentaire dans <code>analyticsController</code>)	Il est encore écrit à l'acceptation d'un premier coach (<code>requests.controller</code>) et par <code>clientCoachController</code>	C'est précisément pourquoi le bug de l'écran Analytics ne touchait qu'une partie des clients : le champ est alimenté sur certains chemins et pas sur d'autres. L'incohérence est réelle et le nettoyage reste à faire
5	Bibliothèque de « 390+ » exercices (F4.9)	416 exactement, répartis en 10 groupes	L'exigence est un plancher, il est dépassé. L'annexe H du dossier le corrige déjà
6	Chronomètre de repos « durées prédéfinies, boutons pause et passer » (brief de la figure retirée)	La durée vient du <code>restTime</code> de l'exercice ; il n'y a pas de durées prédéfinies ni de pause — seulement « Passer »	Le brief décrivait une intention, l'écran fait plus simple. La durée prescrite par le coach vaut mieux qu'un choix générique
7	Enregistrement d'une séance présenté sans réserve (8.2)	<code>deleteMany</code> + <code>createMany</code> hors transaction, avec cascade sur les validations du client	Défaut réel, identifié en préparant cette présentation, avec un correctif précis. Le reconnaître vaut mieux que de le laisser trouver

9 Fiche de survie

9.1 Dix fichiers à savoir citer

Chemin	Ce qu'il porte
<code>backend/src/server.js</code>	Ordre des middlewares, montage des 25 routes, démarrage des crons
<code>backend/src/middlewares/auth.js</code>	<code>authenticate</code> , <code>authorize</code> , <code>authenticateEvenIfSuspended</code>
<code>backend/src/utils/authorization.js</code>	Les cinq règles d'appartenance
<code>backend/src/controllers/appointment.controller.js</code>	Conflit, RRULE, règle de confirmation — le plus dense
<code>backend/src/controllers/sessionController.js</code>	<code>upsertSession</code> et <code>validateSession</code>
<code>backend/src/utils/parseSetData.js</code>	<code>parseNumericField</code> , <code>estimate1RM</code> , <code>getISOWeek</code> — 42 tests
<code>backend/prisma/schema.prisma</code>	36 modèles, 16 énumérations, contraintes et cascades
<code>mobile/src/screens/SessionDetailScreen.js</code>	Validation des séries, les deux chronomètres
<code>mobile/src/services/api.js</code>	Cache du jeton, intercepteurs, 23 modules d'appel
<code>dashboard/.../sessions/_components/session-editor.tsx</code>	Glisser-déposer, validation, enregistrement

9.2 Huit chiffres à ne pas hésiter

Chiffre	Ce qu'il mesure	Chiffre	Ce qu'il mesure
36	modèles Prisma	214	tests, 21 fichiers
25 / 133	modules de routes / endpoints	19 ms	trajet API → base
27	controllers	245 ms	p95 le plus lent, objectif 300
37	relations en cascade	3	tâches planifiées

9.3 Trois démonstrations qui ne peuvent pas rater

Ce qu'on montre	Le chemin, et le point à souligner
Validation d'une série	Mobile → séance du jour → cocher une série. Souligner : la valeur prescrite pré-remplie, le chrono de repos qui démarre seul, le compteur « n / total »
Glisser-déposer accessible	Dashboard → séance → réordonner au clavier . C'est le geste qui prouve le <code>KeyboardSensor</code> , et peu de candidats le montrent
Cache observable	<code>GET /api/notifications/unread-count</code> deux fois : <code>cached: false</code> puis <code>cached: true</code> . Le champ existe exactement pour cette démonstration

Avant la démonstration — réveiller le projet Supabase s'il s'est suspendu (plan gratuit : 7 jours d'inactivité), puis appeler `/api/health` une fois pour sortir les machines Fly de leur veille. Sans cela, le premier écran met plusieurs secondes à répondre.

La phrase à garder pour la fin

Si une question porte sur un défaut — la cascade qui efface les validations, le polling, le jeton en `localStorage`, le test manquant — la meilleure réponse suit toujours la même forme : **nommer le défaut, dire comment je l'ai trouvé, donner le correctif précis, et dire pourquoi il n'est pas fait**. C'est ce que le dossier fait sur la latence de 14.5, et c'est le passage dont un jury se souvient.